



Git Cheat Sheet

The essential Git commands

Creating Snapshots

Initializing a Repository

- `git init`

Staging Files

- `git add file1.js` — Stages a single file
- `git add file1.js file2.js` — Stages multiple files
- `git add *.js` — Stages using a pattern
- `git add .` — Stages the current directory and all contents

Viewing Status

- `git status` — Full status
- `git status -s` — Short status

Committing

- `git commit -m "Message"` — One-line message
- `git commit` — Opens editor for long message

Skipping Staging Area

- `git commit -am "Message"`

Removing Files

- `git rm file1.js` — Removes from working directory + staging
- `git rm --cached file1.js` — Removes from staging only

Renaming / Moving Files

- `git mv file1.js file1.txt`

Viewing Staged / Unstaged Changes

- `git diff` — Unstaged changes
- `git diff --staged` — Staged changes
- `git diff --cached` — Same as above

Viewing History

- `git log` — Full history
- `git log --oneline` — Summary
- `git log --reverse` — Oldest → newest

Viewing a Commit

- `git show 921a2ff` — Specific commit
- `git show HEAD` — Last commit
- `git show HEAD~2` — Two commits before HEAD
- `git show HEAD:file.js` — File version in last commit

Unstaging Files

- `git restore --staged file.js`

Discarding Local Changes

- `git restore file.js` — Restore file
- `git restore file1.js file2.js` — Restore multiple
- `git restore .` — Discard all local changes
- `git clean -fd` — Remove untracked files

Restoring Earlier Version

- `git restore --source=HEAD~2 file.js`

Browsing History

Viewing History

- `git log --stat` — Modified files
- `git log --patch` — Patches

Filtering

- `git log -3` — Last 3 commits
- `git log --author="Michael"`
- `git log --before="2020-08-17"`
- `git log --after="one week ago"`
- `git log --grep="GUI"` — Message contains GUI
- `git log -S"GUI"` — Patch contains GUI
- `git log hash1..hash2` — Range
- `git log file.txt` — Commits touching file

Formatting Output

- `git log --pretty=format:"%an committed %H"`

Aliases

- `git config --global alias.lg "log --oneline"`

Comparing Commits

- `git diff HEAD~2 HEAD`
- `git diff HEAD~2 HEAD file.txt`

Checking Out

- `git checkout dad47ed` — Checkout commit
- `git checkout master` — Return to branch

Finding Bad Commits (Bisect)

- `git bisect start`
- `git bisect bad`
- `git bisect good ca49180`

- `git bisect reset`

Finding Contributors

- `git shortlog`

File History

- `git log file.txt`
- `git log --stat file.txt`
- `git log --patch file.txt`

Finding Author of Lines

- `git blame file.txt`

Tagging

- `git tag v1.0`
- `git tag v1.0 5e7a828`
- `git tag`
- `git tag -d v1.0`

Branching & Merging

Managing Branches

- `git branch bugfix`
- `git checkout bugfix`
- `git switch bugfix`
- `git switch -C bugfix`
- `git branch -d bugfix`

Comparing Branches

- `git log master..bugfix`
- `git diff master..bugfix`

Stashing

- `git stash push -m "New tax rules"`
- `git stash list`
- `git stash show stash@{1}`

- `git stash apply 1`
- `git stash drop 1`
- `git stash clear`

Merging

- `git merge bugfix`
- `git merge --no-ff bugfix`
- `git merge --squash bugfix`
- `git merge --abort`

Viewing Merged Branches

- `git branch --merged`
- `git branch --no-merged`

Rebasing

- `git rebase master`

Cherry Picking

- `git cherry-pick dad47ed`

Collaboration

Cloning

- `git clone url`

Syncing

- `git fetch origin master`
- `git fetch origin`
- `git fetch`
- `git pull`
- `git push origin master`
- `git push`

Sharing Tags

- `git push origin v1.0`
- `git push origin --delete v1.0`

Sharing Branches

- `git branch -r`
- `git branch -vv`
- `git push -u origin bugfix`
- `git push -d origin bugfix`

Managing Remotes

- `git remote`
- `git remote add upstream url`
- `git remote rm upstream`

Rewriting History

Undoing Commits

- `git reset --hard HEAD^`
- `git reset --soft HEAD^`
- `git reset --mixed HEAD^`

Reverting

- `git revert 72856ea`
- `git revert HEAD~3`
- `git revert --no-commit HEAD~3`

Recovering Lost Commits

- `git reflog`
- `git reflog show bugfix`

Amending

- `git commit --amend`

Interactive Rebase

- `git rebase -i HEAD~5`